

ZAUTH

Cloud Integration Architecture

Technical Document

Purpose

Explain how the publicly documented zauth platform connects applications, APIs, identity services, telemetry, x402 services, and cloud-hosted infrastructure.

Scope note

This document describes zauth.inc from public product and privacy documentation. It does not claim access to private production topology. Undocumented implementation details are presented only as reference architecture patterns.

1. Executive Summary

zauth is positioned as a security and trust layer for AI agents and x402-enabled services. Public documentation describes a live registry of x402 endpoints, continuous endpoint testing, provider telemetry, security scanning, and a multi-model research platform. The “cloud connection” is therefore a distributed service architecture: clients, agents, provider APIs, identity services, hosted infrastructure, messaging, and blockchain-aware services communicate over network APIs.

2. Reference Cloud Architecture

The following logical view shows the major cloud-facing relationships described publicly by zauth. It should be read as a functional architecture, not a diagram of zauth’s private internal network.

Component	Cloud-facing role
Client / AI agent	Calls zauth APIs, checks endpoint health, or submits scans.
zauth API layer	Receives HTTPS requests and exposes directory, scanning, and platform services.
Provider Hub SDK	Observes x402 traffic in a provider application and reports telemetry asynchronously.
Cloud runtime	Privacy documentation names Railway and Fly.io for infrastructure hosting.
Redis	Named for caching and real-time messaging.
Identity providers	Google, GitHub, Bitbucket, and X may be linked for authentication.
Blockchain / x402	Payment-aware endpoints and Solana monitoring integrate with the platform.

AI Agent / Developer	HTTPS / API	zauth API & Platform
Provider Application	SDK telemetry	Provider Hub
Identity Provider	OAuth / token exchange	Identity & Session Layer
x402 Endpoint	Health tests / payment flow	Database & Verification
Cloud Services	Hosting / cache / messaging	Railway / Fly.io / Redis

3. Cloud Connection and Data Flow

3.1 Provider Hub Integration

The Provider Hub is a direct application-to-zauth cloud integration. zauth documents an npm middleware package, @zauthx402/sdk, placed before an application’s existing x402 middleware. The middleware observes traffic while leaving the payment logic in the provider application.

```
import { zauthProvider } from '@zauthx402/sdk/middleware'

app.use(zauthProvider(process.env.ZAUTH_API_KEY))
app.use(x402Middleware(...))
app.get('/api/paid', handler)
```

zauth states that telemetry is reported asynchronously. Architecturally, this keeps monitoring traffic off the latency-critical request path and reduces the chance that a telemetry outage will block the provider's API.

3.2 x402 Endpoint Registry

zauth's Database service publishes a directory of x402 endpoints with fields such as URL, HTTP method, network, price, status, success rate, and verification state. An AI agent can query the directory before paying an endpoint, using zauth as a cloud-hosted verification source.

```
GET https://api.zauth.inc/api/directory
?network=base-sepolia&status=WORKING&verified=true&limit=50
```

3.3 Authentication and Identity

zauth's privacy documentation says users may link Google, GitHub, Bitbucket, or X accounts. In a standard cloud authentication flow, the browser or client is redirected to the identity provider, an authorization result is returned to zauth, zauth verifies the token, and zauth creates its own application session and authorization state.

1. User starts sign-in with zauth.
2. zauth redirects to the selected identity provider.
3. The provider authenticates the user and returns an authorization response.
4. zauth verifies the returned token and linked identity.
5. zauth creates or updates the application session.

4. Security Boundaries

Every cloud integration point is a trust boundary. The most important controls for a zauth-connected deployment are credential protection, scoped identity tokens, telemetry minimization, secure messaging, and resilient failure handling.

Boundary	Recommended control
API keys	Store in environment variables or a secrets manager; never commit them to source control.
OAuth tokens	Use TLS, minimum scopes, and short-lived credentials when supported.
Telemetry	Avoid sending secrets or unnecessary personal data in request/response payloads.
Redis / messaging	Use authenticated, encrypted connections and network restrictions.
Cloud hosting	Use least privilege, environment separation, centralized logging, and patch management.
x402 flow	Validate payment requirements and endpoint responses before trusting a paid service.

5. Deployment Pattern

A provider does not need to move its business logic into zauth. Its API can remain in its own cloud environment while zauth acts as an external security, verification, and telemetry layer.

6. Application receives a user or agent request.
7. zauth middleware observes the request and response metadata.
8. Existing x402 middleware performs payment processing.
9. Telemetry is sent asynchronously to zauth.
10. zauth aggregates health and interaction data in cloud services.
11. Operators inspect health and telemetry in Provider Hub.
12. Agents query zauth before selecting or paying an endpoint.

6. Operational Considerations

Availability: Telemetry failure should not take down the primary API path.

Latency: Keep zauth reporting asynchronous wherever the integration permits.

Secrets: Rotate keys and use managed secret storage in production.

Observability: Track SDK errors, API errors, authentication failures, and queue or message latency.

Privacy: Collect and transmit only the data necessary for verification and monitoring.

Resilience: Use bounded retries and exponential backoff for non-critical telemetry.

7. Python Example: Cloud-to-Cloud Query

A Python service can use the public zauth directory as a conventional HTTPS dependency. The example below filters for verified, working endpoints and applies a short timeout so the calling service does not wait indefinitely.

```
import requests

URL = "https://api.zauth.inc/api/directory"

def get_verified_endpoints(network: str):
    params = {
        "network": network,
        "status": "WORKING",
        "verified": "true",
        "limit": 50,
    }
    response = requests.get(URL, params=params, timeout=10)
    response.raise_for_status()
    return response.json()

print(get_verified_endpoints("base-sepolia"))
```

8. Technical Summary

zauth connects to cloud environments through HTTPS APIs, provider-side telemetry middleware, identity-provider token exchange, hosted runtime services, caching and messaging, and x402-aware verification

services. Public documentation names Railway and Fly.io for infrastructure hosting, Redis for caching and real-time messaging, Helius for Solana monitoring, and third-party identity providers for authentication.

The resulting architecture is loosely coupled: the provider application remains authoritative for its own business logic, while zauth provides external verification, telemetry, endpoint health information, and security services.

9. References

- zauth Documentation — <https://zauth.inc/docs>
- zauth Database Documentation — <https://www.zauth.inc/docs/database>
- zauth Provider Hub Documentation — <https://www.zauth.inc/docs/provider-hub>
- zauth Privacy Policy — <https://zauth.inc/privacy>

Accuracy note: Private service topology, internal data stores, deployment regions, and proprietary control logic are not publicly documented and should be confirmed with zauth before using this as a production implementation specification.